

ATELIER TRANSFORMATION IA — ÉQUIPES TECH

L'IA change le métier des développeurs.

Pas pour les remplacer. Pour les transformer en architectes.

Six jours pour accompagner le passage le plus profond du métier dev en 20 ans. On outille la posture autant que la technique — pour qu'ils y trouvent leur place.

DURÉE

6 jours / 6 à 8 semaines

FORMAT

Distanciel · 4 à 8 devs / cohorte

PRATIQUE

70 % du temps

TARIF

~~12 000 €~~ 9 000 € HT

OFFRE

Lancement — places limitées

ANIMATEUR

Thibault Roussel

LE PASSAGE

On ne tape plus le code. On donne une intention.

NOTRE PROMESSE

Vos développeurs ne ressortent pas avec une nouvelle compétence à empiler. Ils ressortent avec une **posture installée** — celle d'un architecte qui formule, cadre, valide. Et avec les outils pour la tenir au quotidien, dans leur codebase, avec leur équipe.

L'agent traduit une intention en code. Le métier du développeur monte d'un cran — il devient celui qui **formule**, qui **cadre**, qui **valide**. Cette montée d'abstraction repose sur une chaîne précise :

intention → décisions → conventions → spec → tests → implémentation

Chaque couche en amont rend la suivante possible. Sans décisions documentées ni conventions, pas de spec lisible. Sans spec, pas de test objectif. Sans test, pas d'implémentation fiable. **C'est ce qu'on apprend à pratiquer dans le bon ordre.**

Pas un déclassement. Une montée d'un cran.

01

Le métier monte d'un cran.

Coder devenait routinier. Formuler, cadrer, valider — c'est plus exigeant, plus créatif, plus proche du sens.

02

Les pratiques d'ingénierie reprennent leur valeur.

DDD, hexagonal, TDD, deep modules. Pas comme dogmes — comme outils de réduction de contexte pour l'agent.

03

La review devient l'activité différenciante.

L'agent produit, l'humain arbitre. Reviewer du code IA avec une grille — pas à l'intuition.

04

L'intelligence ne coûte plus rien.

On peut explorer, jeter, recommencer. Un changement de nature de l'activité dev — pas une simple accélération.

LA CARTE MENTALE

Cinq axes pour lire le passage.

Chaque équipe est un point dans cet espace à cinq dimensions. Le programme avance sur les cinq simultanément — c'est ce qui le distingue d'une formation à un outil ou à une compétence isolée.

AXE 3

Trois terrains de pratique

Greenfield, legacy, production. Pas un seul modèle pour tout. Le greenfield est un laboratoire, le legacy pèse 80 % du terrain, la production est une nouvelle catégorie d'infrastructure. Trois terrains, trois disciplines.

AXE 1 SUR 5

Quatre paliers d'autonomie. D'IA assistante à boucle semi-autonome.

Le palier d'autonomie répond à la question : **qui décide à quoi ?** À chaque grain de décision (la ligne, le hunk, la PR), c'est soit le développeur, soit l'IA. Quatre paliers progressifs, qu'on installe l'un après l'autre.

PALIER	NOM	À LA LIGNE	AU HUNK	À LA PR
A	IA assistante	Dev	Dev	Dev
B	IA pair programmer	IA propose / Dev valide	Dev	Dev
C	IA contributrice	IA	IA	Dev (review)
D	IA en boucle semi-autonome	IA	IA	IA propose / Dev supervise résultat

Palier A — IA assistante (suggest only)

PALIER A

Suggestions uniquement, le dev valide chaque ligne. L'IA est un correcteur orthographique de code. Le dev est pilote.

OUTILS Copilot inline, Cursor en mode tab. **RISQUE** Sous-utilisation, dev qui n'apprend jamais à laisser l'IA prendre la main.

Palier B — IA pair programmer

PALIER B

L'IA génère des hunks ou fonctions entières, le dev valide chaque hunk. Le dev cadre la demande, l'IA produit, le dev arbitre.

OUTILS Cursor / Claude Code en chat-to-edit. **COMPÉTENCES CLÉS** Décomposition de tâche, prompt cadré, review fine.

Palier C — IA contributrice (PR entière)

PALIER C

L'IA produit une PR entière, le dev review en sortie. **Bascule pédagogique majeure** : la review du code IA devient distincte de la review classique. C'est l'activité qui change radicalement le quotidien d'un senior.

OUTILS Claude Code autonome, agent custom orchestré. **COMPÉTENCES CLÉS** Taxonomie des erreurs IA, biais de plausibilité, checklist IA-spécifique.

Palier D — IA en boucle semi-autonome

PALIER D

L'agent itère seul (planifie, code, teste, corrige). Supervision sur le **résultat** et les **métriques**, pas sur les étapes intermédiaires.

PÉRIMÈTRE Un module ou une slice cadrée du code client.

GARDE-FOUS NON NÉGOCIABLES Branche dédiée jamais sur main · Cap de coût par run · Timeout dur · Review humaine obligatoire · Rollback testé une fois pour de vrai.

Critères de bascule entre paliers

- **A → B** : le dev a un `claude.md` personnel fonctionnel et a accepté/rejeté consciemment 20+ hunks IA.
- **B → C** : le dev a produit une PR de bout en bout en mode pair, avec tests, qui passe la review d'un pair humain sans modification majeure.
- **C → D** : l'équipe a un périmètre cadré, des invariants métier couverts par tests, un codebook synthétique, et les garde-fous palier D installés.

Pourquoi c'est un axe et pas une compétence

Le palier d'autonomie n'est pas un outil ni une posture — c'est **la décision principale** qu'une équipe prend, par tâche et par contexte. La même équipe peut être en palier B sur le code de paiement, en C sur les CRUD, en D sur les renames. Choisir le palier juste, c'est l'arbitrage permanent du métier qui change.

OÙ ON VOIT ÇA DANS LE PROGRAMME

L'axe est traversé du début à la fin. **Jour 1** introduction du palier A et premier passage *suggest only*.

Jour 2 et **Jour 3** consolidation du palier B sur hunks puis greenfield. **Jour 4** palier B en contexte legacy — le plus exigeant. **Jour 5** bascule au palier C avec la grille de review IA-spécifique.

Jour 6 mise en place d'une boucle semi-autonome au palier D, avec ses garde-fous non négociables.

AXE 2 SUR 5

L'IA dans le cycle de dev. Où elle aide, où elle ment.

Le cycle de dev moderne n'est pas linéaire — mais ses étapes décrivent des activités distinctes, avec des outils, des paliers d'autonomie dominants et des pièges propres. Le ROI réel de l'IA dépend autant du palier que de l'étape. **Une équipe qui n'utilise l'IA qu'au codage rate la moitié des gains.**

PHASE	ÉTAPE	PALIER DOMINANT
Amont	1. Spec & désambiguïsation	B → C
	2. Recherche & exploration	C
	3. Conception & design	B
	4. Architecture & décisions	A → B
Développement	5. Itération & codage	B → C, montées en D
	6. Tests	B → C
	7. Review	A (par grille humaine)
	8. Debug & investigation	B → C
	9. Documentation	C → D
Legacy	10. Refactor & caractérisation	B → C
Opérationnel	11. Sécurité	A → B
	12. Déploiement & CI/CD	B → C
	13. Observabilité & supervision	B → C

La phase amont — le retour en grâce

Avant qu'une ligne de code soit écrite, l'IA peut être un **partenaire de cadrage** qui interroge le besoin, fait remonter les ambiguïtés non résolues, propose plusieurs designs alternatifs en parallèle. Le passage *besoin* → *spec* → *décomposition* devient un dialogue, pas un monologue. **Piège dominant** : les sorties IA sont plausibles et complètes en apparence, mais lisses sur les cas limites — précisément ceux qui contiennent les bugs et les choix coûteux.

La phase développement — le cœur du gain mesurable

C'est là que les paliers A → D prennent tout leur sens. Génération de code à différents grains, tests, review, debug. **Piège dominant** : l'augmentation de la vitesse masque la dégradation de la qualité — pendant un trimestre. La courbe classique : +30 % de PRs mergées au premier mois, +50 % de bugs en production trois mois après. La vitesse n'est *jamais* une métrique seule.

La phase legacy — le terrain qui pèse 80 % du temps

Les LLMs ont moins d'exposition à *cette* codebase qu'à du code générique. Les conventions implicites passent à côté. Les outils de capitalisation (codebook synthétique, characterization tests) deviennent **nécessaires**, pas optionnels. **Piège dominant** : refactor qui efface des cas limites. L'IA « nettoie » un `if (user.legacyFlag === 'A_2018')` sans réaliser que c'est dix ans de bugfix encodé.

La phase opérationnelle — la nouvelle infrastructure

Sécurité, déploiement, supervision. C'est là que la boucle semi-autonome (palier D) opère réellement. **Piège dominant** : le coût qui explose quand un agent boucle sans cap, et la dérive d'autonomie quand un agent qu'on laisse en D sans audit régulier dévie progressivement de sa mission.

LA RÈGLE D'OR TRANSVERSALE

L'IA accélère la formation d'une opinion. Elle ne valide jamais une décision. Cette règle s'applique à toutes les étapes — particulièrement aux décisions structurantes (architecture, choix de stack, choix de modèle de données) qui se mesurent à 6-18 mois et sur lesquelles l'IA n'a aucun feedback de justesse.

OÙ ON VOIT ÇA DANS LE PROGRAMME

Cet axe traverse tous les modules. **Jour 2** couvre la spec et la conception. **Jour 3** et **Jour 4** sont centrés sur l'itération et le refactor. **Jour 5** est dédié à la review (étape 7) — la seule qui exige de redescendre au palier A. **Jour 6** couvre les étapes opérationnelles 11-13.

AXE 3 SUR 5

Trois terrains, trois pratiques. Greenfield, legacy, production.

Le périmètre détermine plus de choses qu'on ne le croit : palier acceptable, formes des problèmes, mitigations utiles, métriques pertinentes. Un même problème (par exemple la surcomplexité) prend une forme et une gravité radicalement différentes selon le périmètre. **Mêmes mots, conséquences disjointes.**

C'est aussi la première question d'un CTO. « *Sur quel terrain ?* » précède « *quel palier ?* » — il filtre tout le reste.

TERRAIN 1**Greenfield**

« *tout est encore possible* »

Le laboratoire. Code neuf, pas de dette. L'IA produit du code de qualité raisonnable rapidement parce qu'elle n'a pas à comprendre une codebase préexistante. Paliers C et D accessibles plus tôt.

Mais ce n'est pas le terrain représentatif. On ne fait pas du greenfield parce que c'est ce que vous ferez au quotidien. On en fait parce que c'est le seul endroit où on peut observer proprement ce qui se passe.

TERRAIN 2**Legacy**

« *là où votre équipe passe l'essentiel de son temps* »

Le terrain qui pèse 80 % du temps des équipes établies.

Les gains réels d'IA se mesurent ici, pas en greenfield. Une équipe qui ne sait pas faire IA + legacy n'a pas appris l'essentiel.

Le plus dur : LLMs moins exposés à *cette* codebase, conventions tacites qui passent à côté, modularité imparfaite.
Mitigation centrale : caractériser avant de toucher.

TERRAIN 3**Production**

« *le passage qui change le métier* »

Une nouvelle catégorie d'infrastructure. Pas du codage assisté — de l'opération d'agents. L'IA exécute des tâches déléguées dans des branches dédiées, derrière des garde-fous.

Palier D **exclusivement** avec garde-fous non négociables. Tout manque de garde-fou = retour immédiat en C.

Les pièges propres à chaque terrain

TERRAIN	FORME TYPIQUE DES PROBLÈMES
Greenfield	Surcomplexité initiale (architectures sur-dimensionnées) · Cargo-culting (Repository / Factory imposés sans nécessité) · Conventions inconsistantes · <i>Faux sentiment de maîtrise</i> — « <i>si ça marche en greenfield, ça marchera ailleurs</i> ». <i>C'est faux.</i>
Legacy	Refactor qui efface des cas limites (bugfix de 10 ans encodé en silence) · Hallucinations sur les conventions · Régressions silencieuses · Contexte trop volumineux qui noie l'IA
Production	Coût qui explose (agent en boucle sans cap) · Sécurité dégradée (secrets en dur, dépendances vulnérables) · Confidentialité (code propriétaire envoyé hors politique) · Drift d'autonomie sans audit régulier

La discipline du *chaque PR* laisse le code plus orchestrable

Sur les trois terrains, mais surtout en legacy, chaque PR IA touche le code et — bien faite — le laisse dans un état plus orchestrable qu'avant. Plus le code est orchestrable, plus les PRs suivantes coûtent peu et sont fiables. **Compounding positif.**

L'inverse existe aussi : chaque PR IA mal faite ajoute une convention silencieusement violée, une fonction dupliquée, un test tautologique. La courbe descend en silence pendant deux trimestres avant que personne ne s'en aperçoive. **Compounding négatif.** Le programme installe le bon sens d'évolution.

LE PIÈGE DU LABORATOIRE

Le greenfield est le terrain où l'IA brille. C'est aussi celui où l'on prend des mauvaises habitudes par sur-confiance. **Discipline imposée dans le programme** : appliquer en greenfield les mêmes garde-fous qu'on appliquerait en legacy. Les automatismes pris en greenfield sont ceux qui resteront en legacy.

OÙ ON VOIT ÇA DANS LE PROGRAMME

Jour 3 est dédié au greenfield (kata fil rouge orchestré). **Jour 4** est dédié au legacy (audit, characterization tests, codebook). **Jour 6** est dédié à la production (boucle semi-autonome déployée avec garde-fous, méta-observation, gouvernance). Les trois terrains sont travaillés **dans cet ordre précis** : observer en laboratoire, appliquer sur le réel, déployer en infrastructure.

AXE 4 SUR 5

Quatre boucles d'amélioration. L'équipe qui progresse seule.

POURQUOI CET AXE EST CENTRAL

Une équipe qui sort d'un programme de formation classique, six mois plus tard, est *moins* compétente que le jour J. La compétence non utilisée s'érode. Le compounding est l'inverse : **ce qu'on installe doit faire progresser l'équipe d'elle-même**. Sans formateur. Sans budget. Sans dépendance externe. C'est ce qui sépare une formation jetable d'une transformation qui tient.

Quatre boucles distinctes, chacune avec son mécanisme propre, son rituel et sa mesure de progrès. Elles ne sont pas équivalentes : on peut en activer une sans les autres, ou les activer mal. **Le programme cherche à activer les quatre simultanément.**

01 Boucle Agent

L'agent qu'on utilise s'améliore par l'usage : prompts mieux ciblés, slash commands accumulés, skills réutilisables, hooks installés, `claude.md` affiné, MCPs branchés. Plus on utilise, plus on configure ; plus on configure, plus l'agent devient compétent **sur ce projet précis**.

RITUEL CENTRAL

Revue hebdo des prompts qui ont marché et ratés (15 min, vendredi). Ceux qui marchent → skill ou slash command.

02 Boucle Codebase

Le code lui-même devient progressivement **plus orchestrable**. Modularité accrue, tests qui couvrent les invariants, conventions documentées, dette caractérisée. Chaque PR IA touche le code et, bien faite, le laisse plus orchestrable qu'avant.

RITUEL CENTRAL

Mesure baseline → intermédiaire → finale. Discipline du *chaque PR laisse le code plus orchestrable* formalisée en checklist.

03 Boucle Observabilité

On instrumente la **chaîne de fabrication** comme on instrumente la prod. Logs des interactions agent, traces des PRs IA (modèle, coût, temps, taux d'acceptation), méta-patterns d'échec. Ces données informent l'amélioration des autres boucles.

RITUEL CENTRAL

Méta-debrief hebdo de 15 min : qu'est-ce qui a marché, qu'est-ce qui a raté, qu'est-ce qu'on apprend ? Audit aléatoire d'une PR IA sur dix par senior tournant.

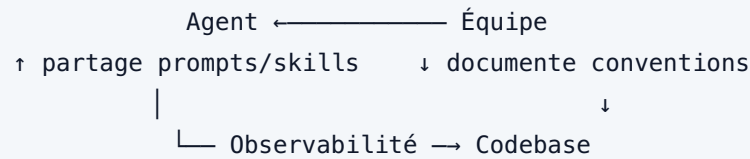
04 Boucle Équipe

L'équipe apprend des uns des autres. Chaque dev ramène ses prompts, ses échecs ; l'équipe fusionne, débriefe, ajuste les conventions. La compétence individuelle alimente la compétence d'équipe, qui à son tour relève chaque individu. **La plus humaine, la plus difficile à installer.**

RITUEL CENTRAL

Méta-debrief hebdo en équipe (15-30 min). Onboarding IA explicite pour les nouveaux. Échange croisé inter-équipes en post-formation.

Comment les quatre boucles se nourrissent



- Sans **observabilité**, les trois autres tournent à l'aveugle. *On améliore quoi ? On ne sait pas.* C'est la boucle la plus structurellement importante.
- Sans **équipe**, les améliorations restent individuelles. Les boucles agent et codebase plafonnent au niveau du dev le plus discipliné.
- Sans **agent**, on n'a pas de levier d'automatisation. Les autres boucles existent mais ressemblent à de l'ingénierie classique.
- Sans **codebase**, le terrain reste hostile. Les agents ne peuvent pas opérer efficacement.

Anti-pattern central : la boucle individuelle

Une boucle agent dégradée par individualisme : chaque dev a sa config locale qui ne se partage pas. Quand un dev part, ses commands partent avec lui. *Hook commercial originel* : « *tu n'as pas une équipe qui utilise l'IA, tu as N développeurs qui l'utilisent chacun dans leur coin* » — c'est la boucle équipe qui répare ça.

Une seule boucle = compounding faible. Quatre = compounding qui se renforce.

Les quatre boucles activées simultanément créent un effet de levier. Une boucle isolée s'éteint. Deux ou trois boucles cohabitent à coût marginal. **Quatre boucles s'auto-entretiennent et amplifient le ROI de chaque PR IA.**

OÙ ON VOIT ÇA DANS LE PROGRAMME

Toutes les boucles sont activées progressivement. **Jour 1** baseline mesurée + carte d'orchestrabilité (boucle codebase, boucle observabilité). **Jour 2** c l a u d e . m d d'équipe (boucle agent, boucle équipe) + modularité comme discipline (boucle codebase). **Jour 5** journal d'erreurs détectées (boucle observabilité). **Jour 6** gouvernance d'équipe + observabilité des appels agent (les quatre boucles consolidées). En inter-sessions : rituels installés (revue hebdo prompts, méta-debrief, capitalisation).

AXE 5 SUR 5

Six modes d'échec. Six grilles de vigilance.

Six catégories regroupent les ~22 problèmes structurels observés en pratique quand une équipe utilise l'IA pour produire du code. Pour chaque problème : un symptôme, une position dans les quatre autres axes (étape x palier x périmètre x boucle dégradée), et une mitigation que le programme installe. **C'est le seul axe qui se croise systématiquement avec les quatre autres.**

MODE 1 Intention La sortie ne fait pas ce qu'on voulait. L'IA optimise pour la lettre de la demande, pas pour le but. <i>Divergence vs vision · Régressions silencieuses · Spec floue → hypothèse cristallisée</i>	MODE 2 Qualité de codebase La sortie dégrade le code. Surcomplexité, duplication, conventions tacites violées, réécritures invasives. <i>Surcomplexité / over-engineering · Cargo-culting · Duplication silencieuse · Conventions implicites violées · Réécritures invasives</i>	MODE 3 Véracité La sortie ment ou invente. Hallucinations confiantes — d'API, de libs, d'auteurs, de versions. <i>Hallucinations d'API · Conseils sécurité confiants et faux · Doc qui dérive du code · Sources fabriquées</i>
MODE 4 Confiance déplacée La sortie n'est pas vérifiable. Tests qui passent sans rien tester, code « qui a l'air bon » qui passe en review à l'intuition. <i>Tests tautologiques · Plausibilité confondue avec correction · Review à l'intuition</i>	MODE 5 Opérationnel Le système qui exécute déraille. Coût qui explose, sécurité dégradée, dépendances fragilisées. <i>Coût en boucle · Fuites de secrets · Dépendances vulnérables · Drift d'autonomie</i>	MODE 6 Humain L'équipe se dégrade. Atrophie senior, dépendance junior, ressenti +20 % / mesuré -20 %. <i>Atrophie senior par fatigue de review · Dépendance junior à l'outil · Biais de plausibilité / autorité · Divergence ressenti / mesuré</i>

Pourquoi chaque problème occupe une position dans l'espace

Un mode d'échec n'est jamais flottant. **Régressions silencieuses**, par exemple, est un problème de catégorie 1 (Intention) qui apparaît à l'étape 5 (Codage) ou 10 (Refactor), au palier B → D, sur du legacy critique, et qui dégrade les boucles codebase et observabilité. La mitigation ? Characterization tests avant tout refactor non trivial.

Cette position dans l'espace est ce qui permet de choisir **la bonne mitigation**. Les mitigations génériques (« faites du TDD ») ne marchent pas. Les mitigations situées dans l'espace marchent.

LA TAXONOMIE TECHNIQUE DES 9 ERREURS IA

Distincte de la grille macro à 6 catégories ci-contre, une **taxonomie technique des 9 erreurs IA** est enseignée au Jour 5 comme outil opérationnel de review. Elle dit où chercher dans le code, à quels symptômes prêter attention, comment les distinguer d'erreurs humaines classiques. La grille à 6 catégories organise la conversation CTO ; la taxonomie à 9 erreurs guide la pratique de review.

Croisements typiques avec les autres axes

- **Mode 2 (qualité)** est dominant en greenfield au palier B → D, dégrade la boucle codebase. Mitigation : *discipline du minimum viable code*.
- **Mode 3 (véracité)** est dominant aux étapes 1, 2, 9, dégrade la boucle agent. Mitigation : *vérification systématique par doc primaire*.
- **Mode 5 (opérationnel)** est dominant en production au palier D, dégrade la boucle observabilité. Mitigation : *cinq garde-fous non négociables*.
- **Mode 6 (humain)** est transverse à toutes les phases, dégrade la boucle équipe. Mitigation : *gouvernance d'équipe documentée + rituels de méta-debrief*.

OÙ ON VOIT ÇA DANS LE PROGRAMME

L'axe est traité explicitement **Jour 5** avec la grille de review IA-spécifique et la taxonomie des 9 erreurs. Mais chaque mode d'échec est nommé et travaillé au moment où il apparaît dans la pratique : **Jour 2** mode 2 (qualité) et mode 3 (véracité) sur le greenfield. **Jour 4** mode 1 (intention) en legacy. **Jour 6** mode 5 (opérationnel) en production. Le mode 6 (humain) est travaillé dans tous les modules — c'est le seul transverse.

LA CHRONOLOGIE

Six jours, six modules. Six à huit semaines.

DURÉE	6 jours, 1 jour / semaine sur 6 à 8 semaines (rythme adaptable)
FORMAT	Intra-entreprise · distanciel · 4 à 8 développeurs / cohorte
PRATIQUE	70 % du temps de formation · codebases fournies + travail sur la vôtre dès le Jour 4
PUBLIC	Équipes qui pratiquent déjà l'IA (cherchent méthode) ou équipes qui n'y sont pas encore passées (cherchent cadrage)

JOUR 1 **Cadrage & sens**

Pourquoi la transition. La place du dev qui change. Hiérarchie d'intention. Cartographie d'orchestrabilité de la codebase cible.

Axes touchés : palier A · cycle (étape 4) · périmètre (audit initial) · boucles (baseline mesurée)

JOUR 2 **Fondations IA-natives**

Prompt vs context engineering. Trois couches de contexte (tacite, technique, stratégique). Architecture orientée agent. Tests revisités. Mise en place du `claude.md` d'équipe.

Axes touchés : palier B · cycle (étapes 1, 3, 6) · boucles (agent + équipe)

JOUR 3 **Greenfield — le laboratoire**

Spec-first vs explore-first. Itérations parallèles autorisées. Quand prototyper, quand cadrer. Application des paliers B → C en conditions optimales.

Axes touchés : palier B → C · cycle (étapes 1, 5) · périmètre (greenfield) · modes 2, 3

JOUR 4 **Legacy — le terrain réel**

Audit de codebase, characterization tests (Feathers), refactor guidé par churn × complexité (Tornhill). C'est là que se joue 80 % du temps des équipes établies.

Axes touchés : palier B · cycle (étapes 8, 10) · périmètre (legacy) · modes 1, 4

JOUR 5 **Review & qualité**

La taxonomie des 9 erreurs IA. Construction de la grille de review IA-spécifique. Journal d'erreurs détectées. La compétence qui change le métier du senior.

Axes touchés : palier C · cycle (étape 7) · boucles (observabilité) · modes 1-6 (transverse)

JOUR 6 **Production, perf, multi-agent**

Observabilité LLM + applicative, méta-observation, cinq garde-fous non négociables, multi-agent (et quand *ne pas* l'utiliser). Gouvernance d'équipe formalisée.

Axes touchés : palier D · cycle (étapes 11-13) · périmètre (production) · boucles (les 4) · mode 5

INVESTISSEMENT

Le tarif. Sans détour.

ATELIER COMPLET — COHORTE

~~12 000 €~~ **9 000 € HT**

6 jours sur 6 à 8 semaines · 4 à 8 développeurs · intra-entreprise distanciel · 1 500 € / jour de formation.

Offre de lancement. Tarif valable pour les premières cohortes. Places limitées, ouvert sans contrepartie autre que votre engagement à suivre le programme dans l'ordre prévu.

Variante exécutive

Format condensé **1 à 2 jours** pour CTO et leadership : cadrage des cinq axes, six modes d'échec, posture, arbitrages structurants. Sans la pratique technique. **À partir de 3 000 € HT / jour**, modulable selon le périmètre.

Pour qui — et pas pour qui

Pour vous, si

- + CTO, dirigeant tech, ou tech lead, avec un sponsor direction.
- + Au moins 4 développeurs, à dominante senior.
- + Codebase avec tests, CI, observabilité minimale.
- + Vous voulez outiller vos devs et leur donner leur place.

Pas pour vous, si

- Équipe très junior, sans senior pour porter les conventions.
- Codebase zombie, sans tests ni CI. Commencez par là.
- Vous voulez « l'IA qui code à votre place ».
- Vous cherchez un slide deck pour un comité.

Comment on démarre

01**Appel de cadrage**

30 min, gratuit. On qualifie ensemble. Pas de pitch.

02 · optionnel**Audit codebase**

1 jour, facturé. Recommandé si legacy ou doute prérequis.

03**Proposition**

Format, calendrier, prix fermes sous 5 jours ouvrés.

04**Démarrage cohorte**

Sous 4 à 8 semaines selon disponibilité.

Prêt pour l'appel de cadrage ?

Trente minutes. Pas de pitch. On regarde votre équipe, votre codebase, vos contraintes. À la sortie, vous savez si le programme a du sens pour vous — ou pas.

Réserver un créneau

Thibault Roussel — Automatik by Zenith

thibault@automatik-by-zenith.com · 06 59 14 56 59

transformation-ia.automatik.tools